Module: `Problem 1`

> This module contains two mutually exclusive definitions of a function named `f`.
> Both implementations are recursive and rely on a while-loop that never terminates for most inputs, leading to infinite recursion and eventual `RecursionError`. The second definition overwrites the first, so only the latter is reachable when the module is imported.

Functions

`f(x)` – First Definition

```
def f(x):
    while x > 3:
        f(x + 1)
```

Description:
Recursively calls itself with an incremented argument (`x + 1`) as long as the original `x` is greater than 3. Because `x` is never modified inside the loop, the condition `x > 3` remains true, causing an infinite recursion.

Parameters: - `x` (*int*): The starting integer value that determines whether the recursion is entered. Must be greater than 3 to trigger the loop.

Returns:
`None`. The function does not contain a return statement; execution ends only when a `RecursionError` is raised due to stack overflow.

Examples:

```
# This call will raise RecursionError because the loop never terminates
f(5)  # → RecursionError: maximum recursion depth exceeded
```

Edge Cases / **Notes**: - If `x <= 3`, the while-condition is false and the function returns immediately (`None`). - For any `x > 3`, the function enters an endless recursion because `x` is never decreased or otherwise altered inside the loop. - The function will eventually hit Python's recursion limit and raise a `RecursionError`.

`f(x)` – Second Definition (overwrites the first)

```
def f(x):
    a = []
    while x > 0:
        a.append(x)
        f(x - 1)
```

Description:
Creates a local list `a`, appends the current value of `x` to it, and recursively calls itself with a decremented

argument (`x - 1`) while `x` remains positive. The loop condition never changes because `x` is not updated inside the `while` block, resulting in infinite recursion for any `x > 0`.

Parameters: - `x` (*int*): The starting integer value. Positive values trigger the recursive loop.

Returns:
`None`. Like the first version, there is no explicit return statement; the function exits only via a `RecursionError`.

Examples:

```
# This call will also raise RecursionError due to infinite recursion
f(3)  # → RecursionError: maximum recursion depth exceeded
```

Edge Cases / **Notes**: - If `x <= 0`, the while-condition is false and the function returns immediately (`None`). - For any `x > 0`, the loop never updates `x`, so the recursion never reaches a base case. - The local list `a` is discarded on each recursive call, making the accumulation of values ineffective. - As with the first definition, the function will eventually exceed Python's recursion depth limit and raise `RecursionError`.